

Git daily routine

A walkthrough of Git practices

May Liu, Guillermo Peretz-Algorta

04-15-2025

Table of contents

0.1	1. Fetch Latest Changes from Remote	1
0.2	2. Check Which Branch You're On	2
0.3	3. Compare Your Current Branch with its Remote Counterpart	2
0.4	4. Update Local Master with Latest Remote Changes	2
0.5	5. Switch to Your Working Branch	3
0.6	6. Merge Updated Master into Your Working Branch	3
0.7	7. Resolve Conflicts if Any	4
0.8	8. Stage and Commit Your Work	4
0.9	9. Push Your Feature Branch to Remote	4
0.10	10. (Optional) Open a pull request to merge branches	5

0.1 1. Fetch Latest Changes from Remote

Before you start working, pull in the latest remote changes without modifying your working files.

```
git fetch origin
```

What It Does:

Fetching downloads new data (commits, branches, etc.) from the remote repository, updating your local tracking branches. It does not alter your working branches.

Why It's Important:

Keeping your tracking branches up to date helps you see if there are any new changes (commits, merges, etc.) that might affect your work before you merge.

0.2 2. Check Which Branch You're On

Ensure you're operating in the correct context by confirming your current branch.

```
git branch
```

What It Does:

Lists all local branches with an asterisk indicating the active branch.

Why It's Important:

It's easy to lose track of what branch you're working on. Double-checking prevents confusion and helps avoid making commits on the wrong branch.

0.3 3. Compare Your Current Branch with its Remote Counterpart

A comparison reveals differences between what's in your local branch and what's been pushed to the remote.

```
git log HEAD..origin/your_branch --oneline
```

```
git log origin/your_branch..HEAD --oneline
```

What It Does:

The first command shows commits present in the remote branch that haven't been applied to your local branch.

The second command shows your local commits that haven't yet been pushed to the remote branch.

Why It's Important:

This helps in catching up to remote changes or verifying your work has been successfully pushed. It's a safeguard against merge conflicts and duplicate work.

0.4 4. Update Local Master with Latest Remote Changes

Get the latest 'stable' branch changes from the remote before integrating them into your feature branch.

```
git switch master
```

```
git pull origin master
```

What It Does:

Switches your working branch to master (or main).

Pulls and merges the latest changes from the remote master branch into your local master branch.

Why It's Important:

Ensures that your base branch is current. A regularly updated base minimizes conflicts when merging your feature branch later.

0.5 5. Switch to Your Working Branch

Go back to your feature or working branch where your work is in progress.

```
git switch your_branch
```

What It Does:

Changes your working directory to the specified branch.

Why It's Important:

It ensures that the next commands (e.g., merging) are performed on your feature branch instead of master.

0.6 6. Merge Updated Master into Your Working Branch

Incorporate the latest updates from master into your current work, ensuring compatibility with the current project base.

```
git merge master
```

What It Does:

Merges the master branch changes into your current branch.

Why It's Important:

Integrating frequent updates from master can reduce large, complex merge conflicts later, keeping your branch aligned with the main line of development.

0.7 7. Resolve Conflicts if Any

If merge conflicts occur, you'll need to resolve them manually.

What It Involves:

Editing conflicting files to reconcile differences.

Testing to make sure that the resolutions do not break functionality.

Why It's Important

Proper resolution ensures stability and correctness of your codebase.

0.8 8. Stage and Commit Your Work

Once your changes are ready and conflicts are resolved, save them in your local repository.

```
git add .
```

```
git commit -m "Meaningful commit message"
```

What It Does:

`git add .` stages all modified files.

`git commit` saves the changes with a descriptive message.

Why It's Important:

Structuring commits with clear messages improves project history, easing code reviews and troubleshooting.

0.9 9. Push Your Feature Branch to Remote

Publish your work online so it can be reviewed by your team members.

```
git push origin your_branch
```

What It Does:

Pushes your committed changes to the remote repository (in the “gpa” branch).

Why It's Important:

This step makes your latest changes available to collaborators and integrates your work into the shared repository.

0.10 10. (Optional) Open a pull request to merge branches

Coordinate with your team by formally requesting to merge your changes into the master branch via a pull request (PR).

What It Involves:

Creating a PR on the repository hosting platform (like GitHub, GitLab, or Bitbucket).

Engaging in code review and discussion about your changes.

Why It's Important:

Pull requests facilitate collaborative code review, ensure adherence to coding standards, and help maintain code quality.